

Lab overview

In this lab, I'll troubleshoot a misconfigured Linux application environment and correct any issues I encounter.

Objectives

By the end of this lab, I should be able to do the following:

- Inspect a kiosk environment to make sure that it is set up properly.
- Review the log output from bash scripts to troubleshoot errors.
- Implement retry logic in a bash shell script using a loop.

Scenario

AnyCompany Gym provides 24/7 access to high quality gyms across the globe. The gym manager for each location will sell one of three subscriptions (Basic, Enhanced, or Extreme) to each new member. Based on the subscription tier, it opens up different parts of the gym. The kiosks that the managers use dispense a radio frequency identification (RFID) tag for customers to unlock the gym entrance.

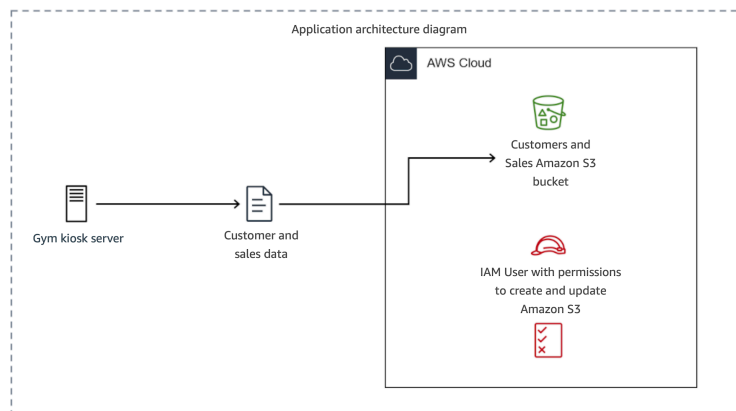


Figure: The gym kiosk server sends customer and sales data files to the AWS Cloud containing an Amazon Simple Storage Service (Amazon S3) bucket. The account includes a role with permissions to create and update Amazon S3.

AnyCompany Gym needs to automate backups and sales reporting in their kiosk system, which is Linux-based. The system stores customer data to a file called `gym-customers.csv`. The file includes the following fields: Customer Number, First Name, Last Name, RFID Tag Number, Gym Access Level, and Expiration Date.

The system stores sales data in a file called `gym-sales.csv`. The file includes the following fields: Transaction Number, Date, Location Number, RFID Tag Number, Customer Number, and Access Type.

AnyCompany Gym wants the sales and customer data uploaded to the cloud every 5 minutes. The company also wants the data backed up locally every 5 minutes to a local backup directory. It wants the backups zipped to save space.

To complete this assignment, I must create multiple scripts and schedule them in the cron table (crontab) schedule. The scripts are as follows:

- *config.sh*: This file contains settings specific to the location, including the location number and S3 bucket name. I can source this file from your other bash scripts.
- *backup.sh*: This script makes copies of the gym-customers.csv and gym-sales.csv files and places them in a backup directory. The script then zips the files into a file named data_backup.csv.
- *upload.sh*: This script uploads the gym-customers.csv and gym-sales.csv files to an S3 bucket in a path under the location ID called labstack-customersalesbucket-/1001/gym-customers.csv and labstack-customersalesbucket-/1001/gym-sales.csv. The numbers 1001 represent the location number.

Only the Linux ec2-user should have read, write, and run permissions on the scripts. All other users should have only read permissions on the created files.

Services used in this lab

AWS CLI

The AWS Command Line Interface (AWS CLI) is a tool that I can use to interact with Amazon Web Services (AWS) services by using commands in your command-line shell. With minimal configuration, I can start running commands from the command prompt in my terminal program that implement functionality equivalent to that provided by the browser-based AWS Management Console.

All AWS infrastructure as a service (IaaS) administration, management, and access functions in the console are available in the AWS API and AWS CLI.

I can explore a service's capabilities with the AWS CLI and develop shell scripts to manage my resources. In addition to the low-level, API-equivalent commands, several AWS services provide customizations for the AWS CLI. Customizations can include higher-level commands that make using a service with a complex API less complicated. An example of this is the aws s3api low-level commands and the higher-level aws s3 commands.

Amazon S3

Amazon S3 is an object storage service designed to offer industry-leading scalability, data availability, security, and performance. Customers of all sizes and industries can store and protect any amount of data for virtually any use case, such as data lakes, cloud-centered applications, and mobile apps. With cost-effective storage classes and easy-to-use management features, I can optimize costs, organize data, and configure fine-tuned access controls to meet specific business, organizational, and compliance requirements.

AWS services not used in this lab

AWS service capabilities used in this lab are limited to what the lab requires. Expect errors when accessing other services or performing actions beyond those provided in this lab guide.

Task 1: Troubleshoot the `upload.sh` script

In this task, I'll correct misconfigurations related to the `upload.sh` script. I'll review the `upload.log` file to find issues with the setup.

Task 1.1: Verify the Amazon S3 bucket name in `config.sh`

I'll connect to the kiosk server and begin investigating the cause of the reported issues. I am initially logged in as the `ssm-user`, but I'm required to switch users to the `ec2-user` to make the necessary changes to the files.

```
sh-5.2$ whoami
ssm-user
sh-5.2$ sudo su ec2-user
[ec2-user@ip-10-0-1-5 bin]$ cd ~/gym-kiosk/logs; pwd
/home/ec2-user/gym-kiosk/logs
```

To view the contents of the `upload.log` file, I'll run this command. I noticed that several entries in the log file look similar to the following excerpt. The most recent entry appears at the end of the file.

```
Uploading sales and customer data for location 1001
S3_BUCKET_NAME=customer-and-sales-bucket
LOCATION_NUMBER=1001
upload failed: gym-kiosk/data/gym-customers.csv to s3://customer-and-sales-bucket/1001/gym-customers.csv An error occurred (NoSuchBucket) when calling the PutObject operation: The specified bucket does not exist
upload failed: gym-kiosk/data/gym-sales.csv to s3://customer-and-sales-bucket/1001/gym-sales.csv An error occurred (NoSuchBucket) when calling the PutObject operation: The specified bucket does not exist
Upload failed. Please check the following:
- Verify that the files 'gym-customers.csv' and 'gym-sales.csv' are present in the '/home/ec2-user/gym-kiosk/data' directory.
- Ensure that the AWS CLI is configured correctly with the appropriate credentials and permissions to access the S3 bucket 'customer-and-sales-bucket'.
- Check the AWS CloudWatch logs for any error messages or additional information about the failed upload.
Please address these issues and try the upload again.
```

The output in the `upload.log` file indicates that the S3 bucket where the process is attempting to upload objects to does not exist. In the next few steps, I'll determine the correct S3 bucket name. The actual bucket name contains `customersalesbucket` in the name.

To retrieve the correct bucket name from the list of buckets in my account, I'll run the following command

```
[ec2-user@ip-10-0-1-5 logs]$ aws s3 ls | grep "customersalesbucket"
2026-07-02 10:33:01 labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp
[ec2-user@ip-10-0-1-5 logs]$
```

Next I need to switch directories into the `bin` folder and then update the `config.sh` script with the correct bucket name.

```
GNU nano 8.3 config.sh
#!/bin/bash
S3_BUCKET_NAME=labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp
LOCATION_NUMBER=1001
```

Even with the bucket name correct, an error occurs when trying to write the objects to Amazon S3. **AccessDenied** indicates that the identity behind the process does not have access to the S3 bucket. In the next task, I'll verify the identity and correct the error.

```
[ec2-user@ip-10-0-1-5 bin]$ ./upload.sh

Uploading sales and customer data for location 1001

S3_BUCKET_NAME=labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp
LOCATION_NUMBER=1001

upload failed: ../data/gym-customers.csv to s3://labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp/1001/gym-customers.csv An error occurred (AccessDenied) when calling the PutObject operation: User: arn:aws:sts::460036325468:assumed-role/SSMRole/i-052944796000faaf4 is not authorized to perform: s3:PutObject on resource: "arn:aws:s3:::labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp/1001/gym-customers.csv" because no identity-based policy allows the s3:PutObject action
upload failed: ../data/gym-sales.csv to s3://labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp/1001/gym-sales.csv An error occurred (AccessDenied) when calling the PutObject operation: User: arn:aws:sts::460036325468:assumed-role/SSMRole/i-052944796000faaf4 is not authorized to perform: s3:PutObject on resource: "arn:aws:s3:::labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp/1001/gym-sales.csv" because no identity-based policy allows the s3:PutObject action

Upload failed. Please check the following:

- Verify that the files 'gym-customers.csv' and 'gym-sales.csv' are present in the '/home/ec2-user/gym-kiosk/data' directory.
- Ensure that the AWS CLI is configured correctly with the appropriate credentials and permissions to access the S3 bucket 'labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp'.
- Check the AWS CloudWatch logs for any error messages or additional information about the failed upload.

Please address these issues and try the upload again.
```

Task 1.2: Configure permissions to load objects into the S3 bucket

In this task, I'll correct the **AccessDenied** issue with the *upload.sh* script. In the AWS environment where the S3 bucket holds the sales and customer data, there is a role user with permissions to create and update S3 objects. The kiosk server must use this user's credentials when interacting with AWS.

To configure the AWS credentials, I'll run this command below and input the expected credentials;

```
[ec2-user@ip-10-0-1-5 bin]$ aws configure
AWS Access Key ID [None]: AKIAWWHCNIBOP3H6C3VD
AWS Secret Access Key [None]: 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
Default region name [None]: us-west-2
Default output format [None]: json
[ec2-user@ip-10-0-1-5 bin]$
```

To run *upload.sh* and verify that the upload is successful, I'll run this command below. I noticed that the output for *upload.sh* contains no errors.

```
[ec2-user@ip-10-0-1-5 bin]$ ./upload.sh

Uploading sales and customer data for location 1001

S3_BUCKET_NAME=labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp
LOCATION_NUMBER=1001

upload: ../data/gym-customers.csv to s3://labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp/1001/gym-customers.csv
upload: ../data/gym-sales.csv to s3://labstack-de822f32-3a00-4ede-a5-customersalesbucket-73wxqoc60kbp/1001/gym-sales.csv

Upload complete!
```

I successfully corrected the errors with [upload.sh](#).

Task 2: Troubleshoot the backup.sh script

In this task, I'll correct misconfigurations related to the *backup.sh* script. I'll review the *backup.log* file to find issues with the setup.

Task 2.1: Correct the backup directory permissions

To view the contents of the **backup.log** file, I'll run this command below. I noticed that several entries in the log file look similar to the following excerpt. The most recent entry appears at the end of the file.

```
Copying customer_data.csv and sales_data.csv to the backup directory...

cp: cannot stat '/home/ec2-user/gym-kiosk/backup/gym-customers.csv': Permission denied
cp: cannot stat '/home/ec2-user/gym-kiosk/backup/gym-sales.csv': Permission denied

Error: Failed to copy files to backup directory. Check permissions.

/home/ec2-user/gym-kiosk/bin/backup.sh: line 24: cd: /home/ec2-user/gym-kiosk/backup: Permission denied
Attempting to zip up the csv files...

    zip warning: name not matched: gym-customers.csv
    zip warning: name not matched: gym-sales.csv

zip error: Nothing to do! (try: zip -r data-backup-2026-07-02-11-06.zip . -i gym-customers.csv gym-sales.csv)
Error: Failed to create backup zip file. Check that the files are present in the backup directory.

Removing the csv files...

rm: cannot remove 'gym-customers.csv': No such file or directory
rm: cannot remove 'gym-sales.csv': No such file or directory

Warning: Failed to remove source files from backup directory.

Backup failed. Please check the error messages and try again.
```

The output indicates that the `cp` command returns a `Permission denied` error when trying to create files in the backup directory. As a result, the remaining commands also fail.

To view the permissions of the backup directory, I'll run this command below. In the output, I noticed that for the backup directory, neither the owner, group, or others have read or write permissions in the directory. This is a misconfiguration. In the next steps, I'll modify the permissions for the backup directory.

```
[ec2-user@ip-10-0-1-150 logs]$ cd ..
ll
total 0
d------. 2 ec2-user ec2-user  6 Jul  3 19:54 backup
drwxr-xr-x. 2 ec2-user ec2-user 57 Jul  3 19:54 bin
drwxr-xr-x. 2 ec2-user ec2-user 52 Jul  3 19:56 data
drwxr-xr-x. 2 ec2-user ec2-user 62 Jul  3 19:56 logs
[ec2-user@ip-10-0-1-150 gym-kiosk]$
```

To update the permissions for the **backup** directory, I'll run the following command. The command modifies the backup directory by adding read, write, and run permissions to the owner, read and run permissions to the group, and read permissions to others.

```
[ec2-user@ip-10-0-1-150 gym-kiosk]$ chmod u+rwx,g+r,o+r backup
[ec2-user@ip-10-0-1-150 gym-kiosk]$
```

To verify that the permissions have been updated, I'll run this command:

```
[ec2-user@ip-10-0-1-150 gym-kiosk]$ ll
total 0
drwxr--r--. 2 ec2-user ec2-user 86 Jul  3 20:16 backup
drwxr-xr-x. 2 ec2-user ec2-user 57 Jul  3 19:54 bin
drwxr-xr-x. 2 ec2-user ec2-user 52 Jul  3 19:56 data
drwxr-xr-x. 2 ec2-user ec2-user 62 Jul  3 19:56 logs
[ec2-user@ip-10-0-1-150 gym-kiosk]$
```

To run the `backup.sh` script, I'll run this command:

```
[ec2-user@ip-10-0-1-150 gym-kiosk]$ cd bin
./backup.sh

S3 Bucket = labstack-de822f32-3a00-4ede-a5-customersalesbucket-wfjcnhbvi5uz
BACKUP_DIRECTORY = /home/ec2-user/gym-kiosk/backup
BACKUP_FILE_NAME = data-backup-2026-07-03-20-18.zip

Copying customer_data.csv and sales_data.csv to the backup directory...

Attempting to zip up the csv files...

updating: gym-customers.csv (deflated 44%)
updating: gym-sales.csv (deflated 60%)

Removing the csv files...

Backup successful.

The backup file is located at /home/ec2-user/gym-kiosk/backup/data-backup-2026-07-03-20-18.zip
```

I successfully corrected the errors with the `backup.sh` script.

Task 3: Add retry logic to the upload script

In this task, I'll modify the `upload.sh` script so that it tries to upload the files to the Amazon S3 bucket **3 times** before exiting.

To delete all of the existing contents from the `upload.sh` script, I'll run the following command:

```
[ec2-user@ip-10-0-1-150 bin]$ truncate --size 0 upload.sh
[ec2-user@ip-10-0-1-150 bin]$
```

The truncate command is used to shrink or extend the size of a file to the specified size. In this case, I'll specify the --size parameter to set the file size to 0 bytes. This deletes all of the contents of the file.

To open upload.sh using the nano text editor, I'll run the following command:

```
GNU nano 8.3 upload.sh
#!/bin/bash
source /home/ec2-user/gym-kiosk/bin/config.sh

DATA_DIRECTORY="/home/ec2-user/gym-kiosk/data"

echo -e "\nUploading sales and customer data for location $LOCATION_NUMBER\n\n"
echo "S3_BUCKET_NAME=$S3_BUCKET_NAME"
echo -e "LOCATION_NUMBER=$LOCATION_NUMBER\n"

for i in {1..3}
do
    # upload sales data to s3 with a $location prefix
    # the && at the end of the next line is a short-circuit operator, it will only run the next line if the previous line succeeded
    # Both lines must succeed for the upload to succeed
    aws s3 cp $DATA_DIRECTORY/gym-customers.csv s3://$S3_BUCKET_NAME/$LOCATION_NUMBER/gym-customers.csv &&
    aws s3 cp $DATA_DIRECTORY/gym-sales.csv s3://$S3_BUCKET_NAME/$LOCATION_NUMBER/gym-sales.csv
    if [ $? -eq 0 ]; then
        echo -e "\n\nUpload complete!\n"
        break
    else
        echo -e "\n\nUpload failed, retrying in 5 seconds...\n"
        sleep 5
    fi
done
```

To test the upload.sh script, I'll run this command:

```
[ec2-user@ip-10-0-1-150 bin]$ ./upload.sh

Uploading sales and customer data for location 1001

S3_BUCKET_NAME=labstack-de822f32-3a00-4ede-a5-customersalesbucket-wfjcnhbvi5uz
LOCATION_NUMBER=1001

upload: ../data/gym-customers.csv to s3://labstack-de822f32-3a00-4ede-a5-customersalesbucket-wfjcnhbvi5uz/1001/gym-customers.csv
upload: ../data/gym-sales.csv to s3://labstack-de822f32-3a00-4ede-a5-customersalesbucket-wfjcnhbvi5uz/1001/gym-sales.csv

Upload complete!
```

I modified the *upload.sh* script to be more resilient by adding retry logic.

Conclusion

I have successfully done the following:

- Inspected a kiosk environment to make sure that it is set up properly.
- Reviewed the log output from bash scripts to troubleshoot errors.
- Implemented retry logic in a bash shell script using a loop.

